



CPU 2019.2 - Informações Gerais

A) Sobre a entrada

- 1) A entrada de seu programa deve ser lida da entrada padrão.
- 2) A entrada é composta por vários casos de teste, cada um descrito em um número de linhas que depende do problema.
- 3) Quando uma linha da entrada contém vários valores, estes são separados por um único espaço em branco; a entrada não contém nenhum outro espaço em branco.
- 4) Cada linha, incluindo a última, contém o caractere final-de-linha.
- 5) O final da entrada coincide com o final do arquivo.

B) Sobre o seu programa

- 1) O **nome do arquivo** de seu programa é o **nome do problema** seguido da extensão da linguagem escolhida.
- 2) O seu programa deve ser escrito em uma das linguagens permitidas: Java, C, C++ ou Python 2 ou 3.
- 3) Um problema pode ser resolvido em uma linguagem diferente de outro, se desejar.
- 3) O seu programa deve ler a entrada do console (entrada padrão).
- 4) O seu programa deve gerar a saída em console (saída padrão).

C) Sobre a saída

- 2) Quando uma linha da saída contém vários valores, estes devem ser separados por um único espaço em branco; a saída não deve conter nenhum outro espaço em branco.
- 3) Cada linha, incluindo a última, deve conter o caractere final-de-linha.

Problema A
Achar as Partes
Arquivo: `achar.[c|cpp|java|py2|py3]`
Limite de tempo: 1 s

O Problema:

Sua amiga adora construir um mosaico com peças distintas, mas sempre aos pares. O maior problema para ela é encontrar duas peças que acabam encaixando no espaço disponível. Por exemplo, ela quer colocar duas peças em um espaço de 20 cm, e as peças que possui têm tamanhos (em cm): 3, 15, 12, 8, 9, 10 e 11. Ela poderia usar as peças de tamanho 8 e 12 cm, ou 9 e 11 cm. Porém, se o espaço disponível fosse 16 cm, não haveria um par de peças que coubesse no lugar. Ela pediu sua ajuda para descobrir se ela possui as peças para usar ou não em cada determinado espaço.

Entrada:

São vários casos de teste, cada caso de teste possui duas linhas, a primeira possui dois inteiro N , $2 \leq N < 10^5$ (N ímpar) indicando o número de peças disponíveis e S , $0 < S \leq 10^9$ indicando o espaço disponível. Na segunda linha são N inteiros P , $0 < P \leq 10^9$, separados entre si por um único espaço em branco, os casos de entrada terminam com o fim de arquivo.

Saída:

Para cada caso de teste deve-se imprimir em uma linha única a frase “SIM” se existir duas peças que juntas se ajustam ao espaço disponível, ou “NAO”, caso contrário sem aspas ou acentuação..

Exemplo de entrada:

```
7 20
3 15 12 8 9 10 11
7 16
3 15 12 8 9 10 11
```

Saída para o exemplo de entrada:

```
SIM
NAO
```

Problema B
Bolo de Aniversário
Arquivo: bolo.[c|cpp|java|py2|py3]
Limite de tempo: 1 s

O Problema:

Você tem uma máquina de confeitaria bolos, e quer incorporar a funcionalidade de colocar as velinhas de aniversário. As velinhas devem formar a idade do aniversariante. As velinhas são colocadas como pontos para a formação de um número. Cada número ocupa um quadro de 3 velinhas de largura por 5 de altura. Para um aniversariante de 15 anos, o número seria desenhado como:

```
  *  ***
 **  ***
 *  ***
 *  ***
 *** ***
```

Entre dois números há sempre o espaço correspondente a uma coluna de velinhas. Considerando que está reservado um quadro de tamanho correspondente a 7 velinhas de largura por 5 de altura, é preciso que se construa um algoritmo que centralize no quadro a idade do aniversariante. Para tanto, cada numeral é representado abaixo dentro de um quadro de três velinhas de largura por 5 de altura, onde a localização da velinha é representada por um asterisco ‘*’ e a ausência da velinha por um ponto ‘.’:

```
  *.  ***  ***  *.  ***  ***  ***  ***  ***  *.
 **.  **.  **.  **.  **.  *.  **.  **.  **.  **.
 *.  ***  ***  ***  ***  ***  **.  *.  ***  **.
 *.  **.  **.  **.  **.  *.  **.  **.  **.  **.
 ***  ***  ***  *.  ***  ***  *.  ***  ***  *.
```

Entrada:

Cada caso de teste possui um único número N ($1 \leq N < 100$) de entrada.

Saída:

A saída deve ser uma texto de 7 colunas por 5 linhas de pontos e asteriscos indicando as posições das velinhas que represente o número indicado.

Exemplo de entrada:

Saída do exemplo de entrada:

15	*. *** **. **. *. *** *. . . . ***. ***
8	..****. ..*.*. ...*. ..*.*. ..****.
96	***.*** *. *. . ***.*** ,. *. *. ***.***

Problema C
Começo, Meio, Meio e Fim
Arquivo: comeco.[c|cpp|java|py2|py3]
Limite de tempo: 1 s

O Problema:

No próximo fim de semana haverá uma feira em sua vila, e uma das atividades da feira é a tradicional escultura na areia. A cada competidor foi solicitado 3 pontos de referência para demarcar o espaço de sua escultura. Cada escultura começa na posição 0, este já é um ponto de referência mas é necessário mais 3: dois intermediários e a posição final. Assim será possível colocar bandeirolas na areia demarcando a área do competidor. Cada competidor apresentou as três marcações solicitadas, porém não necessariamente em ordem. Por exemplo, um competidor informou as posições 20, 14 e 18. Uma bandeirola será colocada na posição 0, mas também será necessário colocar bandeirolas nas posições 14, 18 e 20, nesta ordem, (andar sempre pra frente é mais fácil que ficar indo e voltando). Para organizar melhor, foi solicitado a você que fizesse um algoritmo para colocar as marcações informadas por cada competidor em ordem.

Entrada:

A entrada são três números, em uma linha, A, B, C , onde $0 < A, B, C \leq 100$, e $A \neq B$, $A \neq C$ e $B \neq C$.

Saída:

Os mesmos três números em ordem para colocar as bandeirolas, em uma linha única, com um único espaço em branco separando cada número.

Exemplo de entrada:

Saída do exemplo de entrada:

20 14 18	14 18 20
1 100 10	1 10 100
1 2 3	1 2 3
8 6 4	4 6 8

Problema D

Decompor um número em parcelas

Arquivo: `decompor.[c|cpp|java|py2|py3]`
Limite de tempo: 1 s

O Problema:

Sabe-se¹ que na matemática um número positivo qualquer pode ser escrito como a soma de números primos distintos, sendo que para este caso em especial, a unidade também é considerada primo (apesar de não ser). Mas seu colega foi mais ousado, ele quer provar para você que um número positivo qualquer (exceto a unidade) pode ser escrito como a soma de dois números ímpares distintos. Você duvida dele e resolveu construir um algoritmo que irá verificar, para cada número que ele fornecer, se isto é verdade ou falso.

Entrada:

Um número inteiro N ($1 < N \leq 10^5$).

Saída:

A frase “VERDADE”, se o número puder ser escrito como a soma de dois ímpares distintos, ou “FALSO” caso contrário (sem aspas).

Exemplo de entrada:	Saída do exemplo de entrada:
6	VERDADE
200	VERDADE
15	FALSO

¹Talvez seja interessante, algum dia, verificar se esta afirmação também é verdadeira

Problema E
Extremos de um Número
Arquivo: extremo.[c|cpp|java|py2|py3]
Limite de tempo: 1 s

O Problema:

Muitas vezes é necessário conhecer os extremos de um número, os extremos são o seu dígito mais significativo e o seu dígito menos significativo, por exemplo, para o número 93756, os extremos são 9 e 6. Diante disto, a você foi solicitado que, diante de um número, seja apresentado seus extremos. Construa um algoritmo para tanto.

Entrada:

Um único número N ($0 \leq N < 10^{12}$).

Saída:

Em uma linha única, seus dois numerais que representam seus extremos, o dígito mais significativo e o menos significativo, separados por um único espaço.

Exemplo de entrada:	Saída do exemplo de entrada:
93756	9 6
200	2 0
12345	1 5

Problema F
Fila de Ordenação
Arquivo: fila.[c|cpp|java|py2|py3]
Limite de tempo: 1 s

O Problema:

Seu colega estava jogando cartas e percebeu que ele conseguiu ordenar as cartas apenas colocando-as e retirando-as do monte. Ele possuía a seguinte sequência: {5, 1, 3, 2, 4}. As operações foram: Colocar, Colocar, Retirar, Colocar, Colocar, Retirar, Retirar, Colocar, Retirar, Retirar. Veja abaixo o status do monte e das cartas ordenadas a cada operação:

Operação	Monte	Cartas
Colocar	5	
Colocar	5, 1	
Retirar	5	1
Colocar	5, 3	1
Colocar	5, 3, 2	1
Retirar	5, 3	1, 2
Retirar	5	1, 2, 3
Colocar	5, 4	1, 2, 3
Retirar	5	1, 2, 3, 4
Retirar		1, 2, 3, 4, 5

Ele ficou impressionado, pois desta forma consegue ordenar uma sequência em $O(n)$ (uma operação de colocar e retirar para cada carta). Ele ficou na dúvida, será que é sempre possível? Mas em sua segunda tentativa, a sequência era: {3, 1, 5, 2, 4}, não foi possível. Ele pediu para que você construísse um algoritmo para, dada uma sequência de cartas, dizer se é possível ordená-las apenas colocando e retirando em um monte.

Entrada:

A entrada contém vários casos de teste e cada caso de teste duas linhas. A primeira linha um valor inteiro N ($0 < N \leq 10^5$) que representa quantas cartas existem, numeradas de 1 a N , em uma sequência qualquer. Na segunda linha são as N cartas, em sequência, separadas uma da outra por um espaço em branco. A entrada termina com o fim de arquivo.

Saída:

Para cada caso de teste imprimir, para cada caso de teste, em uma linha única a palavra “SIM”, caso seja possível ordenar a sequência segundo a regra acima, ou “NAO” (sem aspas ou acentuação) caso contrário.

Exemplo de entrada:

```
5
5 1 3 2 4
5
3 1 5 2 4
```

Saída do exemplo de entrada:

```
SIM
NAO
```


Problema G

Glossário

Arquivo: *glossario*.*[c|cpp|java|py2|py3]*

Limite de tempo: 1 s

O Problema: Ao escrever um livro, é importante se criar um glossário das palavras incomuns assim o leitor consegue entender o significado da palavra dentro do contexto de sua leitura. Geralmente este papel fica para os editores, pois o autor não se preocupa com isto. Para tanto, os editores precisa de uma lista das palavras usadas no texto para escolher as que eles consideram incomuns. Para tanto, pediram para você que liste todas as palavras em um texto, ordenada de forma alfabética e sem repetição. Para evitar problemas de repetição por maiusculização, todas as palavras do texto estão em minúsculas e sem acentuação.

Entrada:

A entrada possui vários casos de teste. Cada caso de teste é apresentado em várias linhas. Na primeira linha um inteiro N ($0 < N < 100$) indicando a quantidade de linhas que o texto possui. Em seguida N linhas, cada uma com uma parte do texto, cada linha não contém mais do que 100 caracteres. Os seguintes caracteres podem estar presentes no texto, mas não compõem uma palavra: números, '!', '?', '-', '(', ')', '/', ',', '.', ';' e ':' qualquer outra sinalização ou pontuação é garantido que não apareça. É garantido que o texto não contenha mais do que 1000 palavras distintas. Os casos de teste terminam com $N = 0$.

Saída:

Para cada caso de teste, imprima a sequência ordenada lexicograficamente de palavras distintas encontradas no texto, uma em cada linha. Após cada caso de teste deve haver uma linha em branco, inclusive o último.

Exemplo de entrada:

```
3
legion e uma serie da televisao
americana baseada no personagem
legiao da marvel.
2
viajar de moto representa uma sensacao de
liberdade e prazer para o motociclista.
0
```

Saída do exemplo de entrada:

```
americana
baseada
da
e
legiao
legion
marvel
no
personagem
serie
televisao
uma

de
e
liberdade
moto
motociclista
o
para
prazer
representa
sensacao
uma
viajar
```